

COMP2321: Coursework Overview – “CHESS FRAGMENTS”

This game consists of two players: **Agent**, your designed algorithm, and **Opponent**, which represents a set of predefined baseline algorithms. **Agent** and **Opponent** compete on an 5x5 grid in a turn-based strategy game inspired by chess.

1. Rules:

The board consists of several pieces chosen from the set {**Right**, **Knight**, **Bishop**, **Queen**, **King**, **Pawn**}, each with its own movement rules:

- **Right:** Moves any number of squares horizontally or vertically. They can also move in an L-shape: two squares in one direction, then one square perpendicular. It can jump over other pieces.
- **Knight:** Moves in an L-shape: two squares in one direction, then one square perpendicular, jumping over other pieces.
- **Bishop:** Moves any number of squares diagonally.
- **King:** Moves one square in any direction.
- **Queen:** Moves any number of squares diagonally, horizontally, or vertically.
- **Pawn:** Move one square forward. Pawn captures an opponent piece by moving one square diagonally forward to the occupied square. When pawn can reach the opponent's back rank, pawn is promoted to Queen.

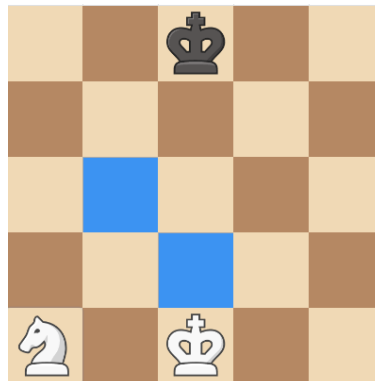
The piece **King** is always present on the board. Other pieces may or may not appear, depending on our board setup. In each game, **Agent** can play the white side, and **Opponent** plays as the black side, or vice versa.

2. Example of piece movements:

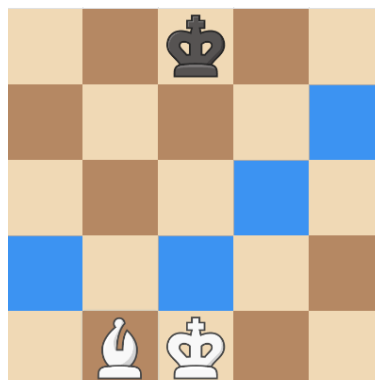
Right (represented as **R** for white and **r** for black)



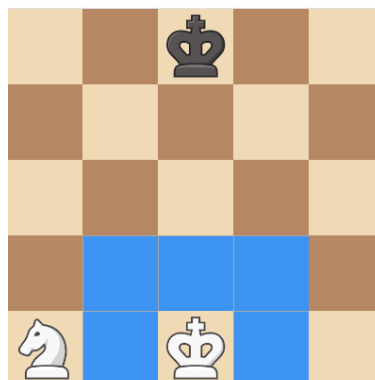
Knight (represented as **N** for white and **n** for black)



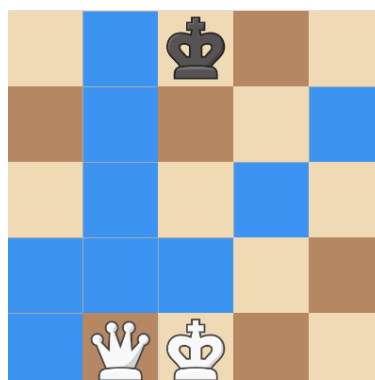
Bishop (represented as **B** for white and **b** for black)



King (represented as **K** for white and **k** for black)

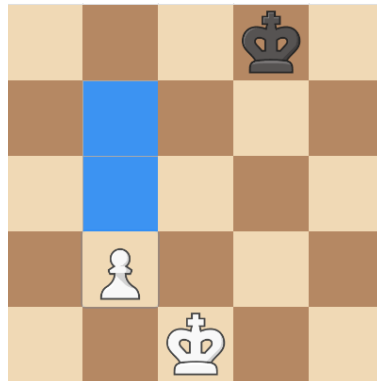


Queen (represented as **Q** for white and **q** for black)

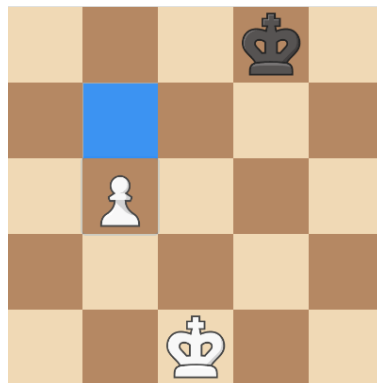


Pawn (represented as **P** for white and **p** for black)

- In their first move, Pawn has the option to move either one square or two squares forward if the path is clear.



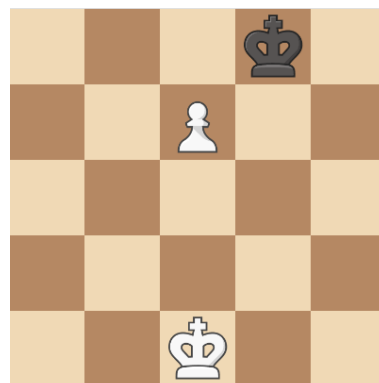
- Starting from their second move, Pawn can only move 1 square forward.



- Pawn captures an opponent piece by moving one square diagonally forward to the occupied square as below:

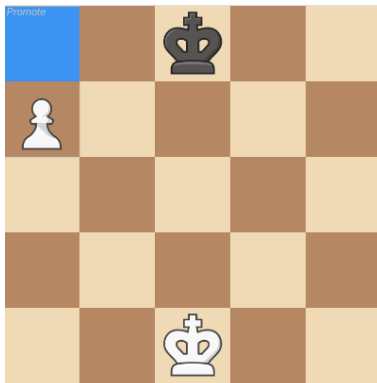


Before moving

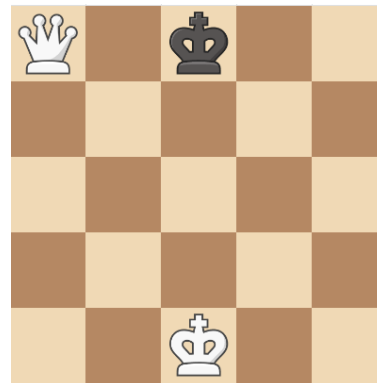


After moving

- When pawn can reach the opponent's back rank, pawn is promoted to Queen as below:



Before moving



After moving

- Similar to standard chess, a pawn can also capture an opponent's pawn via *en passant*.

In general, all available move and capture options (for all pieces) can be determined using the legal move checking function provided in the CW Supporting Package.

3. Board presentation in the Python environment

In the CW support package, the game board is presented by a 2D matrix with a shape of (5,5). Each chess piece on the board is denoted by a single letter: **uppercase letters for white pieces** and **lowercase letters for black pieces**. You can further explore this representation in the provided CW support package.

	0	1	2	3	4
0	r	b	k	q	n
1	p	p	p	p	p
2					
3	P	P	P	P	P
4	N	Q	K	B	R

A matrix shape of (5,5)

4. Win, Lose, or Draw:

Win/Lose: The game is lost (and the opponent wins) under one of the following conditions

- Checkmate: your King is checkmated. In other words, it is attacked with no legal escape.
- Stalemate: it is your turn to move, but you have no legal moves available.
- Thinking timeout: your agent fails to decide their next move within the predefined time limit.
- Illegal move: your agent makes an invalid move during their turn.

Draw: A draw can occur under several condition:

- Only 2 kings remain on the board.

- Fivefold repetition: the game is ended by a draw if the exact board position occurs five times during the game, regardless of whether the repetitions are consecutive or not.
- Game timeout: the game is declared a draw if neither Player 1 nor Player 2 achieves a win or loss within the predefined time limit.

5. Preparing Your Agent (*agent.py*) for Submission

You are required to design an **Agent** (implemented in **agent.py**) following the specified format. The initial positions of the pieces on the game board and **Opponent** are predefined. In Moodle, **Opponent** is implemented with varying difficulty levels, and the board setups also differ in complexity. Therefore, you should design **Agent** to be as smart as possible in order to compete effectively in different settings.

You may consider implementing algorithms such as minimax, alpha–beta pruning, etc. to guide **Agent** decide the best next move. It is recommended to begin with a simple, maybe not-so-clever algorithm to test against **Opponent**, and then gradually move to more sophisticated approaches.

You should also evaluate the computational cost (e.g., the number of nodes expanded) when deciding the next move within the predefined time limit to avoid the *thinking timeout*. This can be done by adjusting the search depth or by applying other optimization techniques, with the goal of producing a solution that is both strong and computationally efficient. Additionally, make sure to monitor the provided thinking time variable to ensure your agent selects the most reasonable move before the *thinking timeout* occurs.

In the CW supporting package, a *basic example* is provided, including a simple **Agent** and **Opponent**, to help you get familiar with the game rules and the required **agent.py** format. Note that the sample **Agent** provided is for illustration purposes only and will **not** be competitive against **Opponent** in the assessment system. You should complete your own **Agent** and save it as **agent1.py** in the required format before submission.

6. Assessment

The assessment has two parts. In Part 1, your mark is determined by how well you do against our **basic Opponent**. (60% mark)

In Part 2, your mark is determined by how well you do against our **competitive Opponent**. (40% mark)

6.1. Part 1: Agent vs Basic Opponent (60%)

Once you develop your own **Agent**, prepare it as **agent.py** in the required format. In Part 1 assessment, your **Agent** will be tested against **basic Opponent** under different difficulty levels varying in both strategy and board setup, starting from the easiest level (**Game1**) to the hardest (**Game6**).

Starting from week 5, a submission link will be provided on Moodle, where you can submit your **agent.py** solution once a week. Your **Agent** will then be tested against our **basic Opponent** across different board settings. The purpose is to give you an opportunity to “warm up” and engage in Part 1 as well as observe how your *work-in-progress* **Agent** behaves in our server

environment which is the platform used for both Part 1 and Part 2 assessments. After few days, you will receive feedback indicating whether each game ended with a win, loss, or draw, along with a log file reporting the game details. Based on this feedback, you may choose to improve your agent and resubmit it for evaluation in the following week. Since these weekly submissions are designed mainly to provide *external feedback* for your *work-in-progress Agent*, you should not expect the **same board setup or same Opponent** to be used in **the Part 1 assessment in week 11**. Weekly submissions from week 5 to week 10 are also optional.

The **deadline for the assessed submission is December 12th (week 11)**. The results of this submission will count as your Part 1 marks, and the same Agent will be used for the Part 2 assessment.

Before the **submission deadline**, you must also submit a **short presentation** (3-5 slides) explaining how your **Agent** strategy works. The marks of your submission are conditioned upon the submission this **short presentation**, and you must be prepared to present your slides to the class if requested in *week 12* (you will be notified the week before if selected to present). Failure to submit the slides, or failure to present when requested, will result in a **30% penalty** applied to your marks.

6.2. Part2: Agent vs Competitive Opponent (40%)

Your **Agent** submitted in Part 1 will be tested against a set of **more competitive Opponents** designed to push its limits and evaluate its full capabilities. This is your chance to demonstrate how well your **Agent** performs in more challenging settings.

In Part 2, your marks will be calculated based on your Agent's overall performance, that is, the number of games it wins, draws, or loses against our competitive Opponent across different settings.

7. Important Things to Consider

1. You are allowed to use **online references** and **GenAI tools** to support developing your Agent solution. However, you are **not permitted to copy** another student's `agent.py` solution. All submissions will be checked for **plagiarism** to detect any similarity.
2. You should thoroughly test your **agent.py** solution in your local machine before submitting that to Moodle. Do not consider the weekly evaluation for debugging code – do that on your local machine. The weekly submissions are for testing that the agent runs correctly in our testing environment.
3. The Moodle system may become overloaded close to the submission deadline, especially in *week 11*, so **do not wait** until the last minute to submit your **agent.py** solution.